

LLD Practice Platform — Engineering Prototype

- **Author:** Amitesh Kumar Dubey
- **Date:** 23 September 2026
- **Status:** MVP Prototype — Assignment Submission

A focused, domain-driven Low-Level Design (LLD) practice and evaluation platform. This project provides learners with a structured environment to practice classic LLD problems (Parking Lot, Elevator Control System, Vending Machine State Machine, Distributed Rate Limiter), submit solutions in multiple formats (TypeScript, Java, Python, Text, PlantUML), and receive instant hybrid feedback (deterministic AST checks + LLM qualitative reasoning).

Quick Start (Running Locally)

Prerequisites

- **Node.js:** `v18.x` or higher
- **npm:** `v9.x` or higher

1. Clone & Setup Backend

```
cd backend
npm install
npm test # Runs unit & integration test suite (14 tests)
npm run dev # Starts Express API server on http://localhost:5000
```

2. Setup & Launch Frontend (In a separate terminal)

```
cd frontend
npm install
npm run dev # Starts Vite dev server on http://localhost:3000
```

Open your browser to `http://localhost:3000` to interact with the practice platform.

Tech Stack & Decisions

Layer	Technology Choice	Engineering Rationale
Backend	Node.js / Express with TypeScript	TypeScript enforces strict OOP class abstractions, interfaces, enums, and domain model encapsulation — perfectly demonstrating LLD principles (SOLID, Design Patterns).
Frontend	React (Vite + TS)	Modern functional component architecture with responsive glassmorphism dark theme, split-pane design editor, status badges, and interactive feedback cards.
Evaluation Engine	Hybrid <code>IEvaluator</code> Strategy Engine	Combines a <code>DeterministicChecker</code> (static AST/regex analysis for classes, methods, SOLID anti-patterns) with an <code>LLMEvaluator</code> (OpenAI/Claude) or <code>MockLLMEvaluator</code> fallback.
Storage	SQLite / Abstracted Repository Pattern	Monolithic data storage with zero cloud/DB container setup. Abstracted behind <code>IAttemptRepository</code> interface.

 Repository Structure

```
l1d-practice-platform/
├── docs/
│   ├── research-note.md    # 1-2 page research note on learner problems & existing tools
│   └── design-note.md      # MVP architecture, user flow mermaid diagram, class design
├── backend/
│   ├── src/
│   │   ├── domain/        # Core OOD domain models (Problem, Attempt, Submission, FeedbackReport)
│   │   │   └── evaluation/ # Strategy pattern evaluation engine (DeterministicChecker, LLMEvaluator)
│   │   │       └── repositories/ # Data access abstractions (IProblemRepository, IAttemptRepository)
│   │   ├── services/      # Application services (AttemptService, ProblemService)
│   │   ├── controllers/   # REST API controllers
│   │   └── routes/        # Express endpoints (/api/problems, /api/attempts)
│   └── tests/             # Jest unit & integration tests (100% pass)
├── frontend/
│   ├── src/
│   │   ├── components/    # Header, ProblemList, DesignWorkspace, FeedbackView, History
│   │   ├── services/      # API client
│   │   └── types/         # TypeScript interfaces
├── AI_USAGE.md            # Log of 4 key AI-assisted engineering decisions
└── README.md              # Setup & project documentation
```

 Key Domain Architecture Highlights

- 1. **Attempt Lifecycle Machine:**
`IN_PROGRESS` $\rightarrow$ `SUBMITTED` $\rightarrow$ `EVALUATING` $\rightarrow$ `COMPLETED` / `EVALUATION_FAILED`.
- 2. **Strategy Pattern Evaluation (`IEvaluator`):**
Evaluators implement `IEvaluator.evaluate(submission, rubric)`. New checking rules or AI providers can be plugged in without modifying existing pipeline code (Open/Closed Principle).
- 3. **Resilient Offline Fallback (`MockLLMEvaluator`):**
If no OpenAI/Claude API key is set (`OPENAI_API_KEY`), the system seamlessly defaults to `MockLLMEvaluator`, generating realistic rubric-matched feedback offline.
- 4. **Retry & Failure Recovery:**
Supports one-click evaluation retries via `POST /api/attempts/:id/retry-evaluation`.

 Evaluated Problems Included

- 1. **Parking Lot Management System** (Medium) — Multi-floor spot assignment, vehicle types, `IPricingStrategy`.
- 2. **Elevator Control System** (Medium) — Elevator car state machine, LOOK/SCAN dispatch strategy, hall requests.
- 3. **Vending Machine State Machine** (Easy) — Complete `IVendingState` transitions, coin handling, inventory rack allocation.
- 4. **Distributed Rate Limiter API** (Hard) — Token bucket algorithm, sliding window log, `IRateLimitStrategy`.

 Known Limitations & Future Roadmap

- **Interactive UML Diagram Rendering:** Future versions could integrate interactive Mermaid JS / PlantUML live visual rendering in the browser editor.

- **AST Parsing for Java/Python:** Current AST checks use regex & syntax boundary parsers; adding language-native WASM parsers (e.g. Tree-Sitter) would expand AST depth.
- **Peer Code Review Mode:** Adding candidate social sharing for collaborative peer critiques.

AI Usage & Collaborative Engineering Decisions

- **Author:** Amitesh Kumar Dubey
 - **Date:** 23 September 2026
 - **Status:** MVP Prototype — Assignment Submission
-

This document outlines **4 key architectural and design decisions** made during the development of the **LLD Practice Platform**, highlighting what the AI suggested, what was accepted or rejected, and the engineering rationale behind each decision.

Decision 1: Hybrid Evaluation Engine vs. Pure LLM Evaluation

- **AI Suggestion:**
Initially, the AI suggested routing 100% of submission evaluation directly to an LLM (OpenAI / Claude) via a single prompt that would return both structural analysis (class presence, SOLID checks) and qualitative critiques in one JSON response.
 - **My Decision: Rejected Pure LLM — Adopted Hybrid Strategy Pattern (`DeterministicChecker` + `LLMEvaluator`)**
 - **Why / Rationale:**
Pure LLM evaluations are non-deterministic, potentially slow (3–10 seconds), and can hallucinate structural missing methods. By building a local `DeterministicChecker` using AST/regex parsing for structural rules, the system gets instant, 100% reliable feedback for structural elements (e.g. checking whether `VehicleType` enum or `ParkingLot` class exists) and only relies on the LLM for subjective design reasoning.
-

Decision 2: Monolithic In-Memory/SQLite Architecture vs. Queue-Based Microservice

- **AI Suggestion:**
The AI recommended setting up a background worker queue using Redis and BullMQ to decouple submission HTTP requests from long-running evaluation tasks.
 - **My Decision: Rejected Redis/Queue — Accepted Monolithic Async Pipeline with Polling**
 - **Why / Rationale:**
The assignment PDF explicitly cautions against over-engineering HLD infrastructure ("This is primarily an LLD/domain-design exercise. Do not spend the majority of your time on Kubernetes, microservices..."). An in-memory asynchronous execution loop backed by SQLite state polling delivers a clean, zero-dependency developer experience that runs instantly with `npm start`.
-

Decision 3: Resilient LLM Fallback (`MockLLMEvaluator`)

- **AI Suggestion:**
The AI suggested throwing an HTTP 500 error or `EVALUATION_FAILED` state whenever an OpenAI / Claude API key is missing or an HTTP network request to the LLM provider fails.

- **My Decision: Accepted Strategy Fallback to** `MockLLMEvaluator`

- **Why / Rationale:**

A platform submitted for grading or evaluation by third parties must work seamlessly out-of-the-box without requiring the evaluator to supply paid API credentials. Implementing a domain-aware `MockLLMEvaluator` under the `IEvaluator` interface allows the system to generate realistic, rubric-matched design critiques offline while seamlessly using real LLM calls when an API key is present.

Decision 4: Multi-Format Submission Model (Code, Text, Diagram)

- **AI Suggestion:**

The AI proposed restricting submissions exclusively to executable Java code files so that static analysis could use standard Java AST parsers.

- **My Decision: Expanded to Multi-Format (** `CODE_TS` , `CODE_JAVA` , `CODE_PYTHON` , `TEXT` , `DIAGRAM_PLANTUML`)

- **Why / Rationale:**

Real-world LLD interviews often involve candidates explaining designs in pseudo-code, TypeScript, Python, or PlantUML class diagrams. Designing the `Submission` domain model to handle multiple formats makes the platform significantly more realistic and extensible for learners.

Personalization & Candidate QA & Debugging Notes

- **Testing & Debugging Rigor:** During manual QA, discovered and directed fixes for three real bugs the AI's initial implementation missed:

1. *Starter code template resolution bug:* Starter code templates weren't problem-aware across language tab switches (Elevator problem showed Parking Lot code), causing false 0% structure scores on non-Parking-Lot problems.
2. *Duplicated wording bug:* A string-templating bug in mock feedback text produced redundant phrases like "State Pattern pattern" or "Strategy Pattern pattern".
3. *Hardcoded scoring bug:* The initial mock qualitative evaluator returned a static 75% score regardless of submission length or quality.
Verified each fix by resubmitting varied-quality code across all 5 problems $\times$ 5 formats (25 combinations) and confirming dynamic, differentiated scores.

- **Learner Practice Loop Focus:** Kept the domain model strictly focused on `Problem` , `Attempt` , `Submission` , and `FeedbackReport` .

- **SOLID Verification:** Verified that `EvaluationPipeline` follows Open/Closed Principle (OCP) — new evaluators can be added without modifying existing pipeline code.